Course 1 Master Revision: Generative AI Applications

Last corrected: 8 August 2026

What matters for jobs

Course 1 is useful for the mental model and framework vocabulary. Interview credibility comes from explaining boundaries, failure modes, and how the concepts appear in the financial extraction project.

1. Generative AI application model

The model is one probabilistic component inside an application. Production responsibility remains with ordinary software:

- Validate inputs and outputs.
- Authenticate users and authorize data/actions.
- Bound time, retries, tokens, and cost.
- Evaluate quality and regressions.
- Trace failures and protect sensitive data.

Project evidence: the `EarningsExtraction` schema and fake malformed response tests establish a typed boundary around model output.

2. Prompt anatomy

A strong request separates:

1. System/developer constraints and role.
2. The task and expected output.
3. Trusted context or retrieved evidence.
4. User input, clearly delimited as untrusted data.
5. Examples only when they improve measured performance.

Zero-shot means no examples. One/few-shot supplies examples that clarify output patterns or edge cases. Examples consume tokens and can bias the model, so evaluate representative cases.

Prompt instructions are not a security boundary. A malicious document can contain indirect prompt injection, and the model cannot authorize its own actions.

3. Structured output

Native JSON-schema/Pydantic output is preferable to parsing free-form text because it reduces syntax and shape failures.

It proves:

- Required fields and types were returned.
- Declared constraints were satisfied.

It does not prove:

- Values are true or present in the source.
- An evidence quote supports the claimed value/unit/period.
- A tool call is authorized or safe.

- The same prompt will always return identical content.

Project evidence: `live_extractor.py` requests an `EarningsExtraction`; `verify_evidence` adds a first source-presence check. The latter still needs claim/evidence consistency validation.

4. Decoding and consistency

Temperature changes the sampling distribution when the provider/model exposes it. Lower temperature often reduces variation. Greedy decoding chooses the highest-probability next token.

Neither "temperature 0" nor greedy decoding should be described as a universal guarantee of identical results: provider infrastructure, model updates, floating-point behavior, and tie-breaking can still matter.

Self-consistency samples multiple reasoning paths and aggregates their answers. It can improve some reasoning tasks, but it costs more and can amplify a shared error. Evaluate it rather than treating it as an accuracy guarantee.

5. LCEL and framework abstractions

LangChain Expression Language composes runnables, commonly as:

```
prompt | model | output_parser
```

A dictionary can fan out named inputs; functions can be wrapped as runnable steps. The job-relevant skill is tracing data types, errors, concurrency, configuration, and observability across the pipeline - not memorizing one version's imports.

Framework APIs change. Isolate them behind project interfaces and verify current official docs when implementing.

6. Chains, workflows, and agents

- Chain: fixed composition of steps.
- Workflow: application-controlled branches/state/retries around known steps.
- Agent: model dynamically selects among constrained actions based on state/observations.

Use the simplest form that meets the requirement. Agents add variability, latency, cost, and security surface.

A ReAct-style loop alternates model reasoning/action selection with tool observations. In production, keep reasoning private, validate tool arguments, authorize outside the model, limit iterations/time/tokens, and define terminal states.

7. RAG versus long context versus fine-tuning

- Prompt/long context: good when the relevant material is small and fits the request budget.
- RAG: good for large, dynamic, private, filtered, or attributable knowledge.
- Fine-tuning: good for learned behavior/style/task patterns, not frequent fact updates.
- Tool/API call: good for authoritative live data or deterministic computation.

RAG can reduce unsupported answers when retrieval and prompting work, but it does not guarantee grounding.

8. Model selection and cost

Start with a capable reference model to establish a quality baseline, then evaluate smaller/cheaper candidates on the same versioned dataset. Route by task only if routing errors, added complexity, latency, and savings are measured.

Evaluate quality, latency percentiles, availability, privacy, region, context limits, structured/tool support, rate limits, model/version stability, and total cost.

Prompt caching savings depend on provider rules and repeated prefix reuse. Semantic caches require tenant isolation, TTL/invalidation, thresholds, and protection against stale or unsafe matches.

Self-hosting can improve control or economics at sufficient scale, but serving improvements depend on workload, hardware, model, and configuration. Benchmark rather than quoting universal gains.

9. Interview drills

1. What does Pydantic prove about an LLM response, and what does it not prove?
2. When would you choose a deterministic workflow over an agent?
3. Compare prompting, long context, RAG, fine-tuning, and tools for one use case.
4. Why can temperature zero still fail to provide reproducibility?
5. How would you evaluate a smaller model before routing traffic to it?
6. How could indirect prompt injection affect a RAG/tool application?
7. Explain one limitation in the current financial extraction prototype.

One-minute project answer

"I built a typed financial extraction prototype using Pydantic and Gemini structured output. The schema removes malformed-output handling from downstream code, but it does not prove the facts. I added a deterministic evidence-presence check and tests for malformed and fabricated evidence, then identified its limitation: a real quote can still be paired with the wrong value or period. My next step is a versioned evaluation set and real retrieval with stable citations, followed by claim-to-evidence consistency checks. That distinction between shape, source presence, and factual support is central to how I design reliable LLM systems."